

Redis issue-5632 분석

1 개요

redis server에 empty string("") key가 있을 때 KEYS "\x00*z"라는 명령을 보내면 redis server 가 크래시 되는 이슈

1.1 재현 명령

```
SET "" ""          # key가 "", value가 "" 인 데이터 추가
KEYS "\x00*z"      # null(\x00), *, z라는 패턴을 갖는 key를 검색
```

2 로그

```
... # 생략
2069120:M 27 Sep 2026 14:46:46.879 - Accepted 127.0.0.1:46186
2069120:M 27 Sep 2026 14:46:48.798 - DB 0: 2 keys (0 volatile) in 4 slots HT.
2069120:M 27 Sep 2026 14:46:48.798 - 1 clients connected (0 replicas), 812064 bytes
in use
2069120:M 27 Sep 2026 14:46:53.818 - DB 0: 2 keys (0 volatile) in 4 slots HT.
2069120:M 27 Sep 2026 14:46:53.819 - 1 clients connected (0 replicas), 812064 bytes
in use
2069120:M 27 Sep 2026 14:46:58.838 - DB 0: 2 keys (0 volatile) in 4 slots HT.
2069120:M 27 Sep 2026 14:46:58.838 - 1 clients connected (0 replicas), 812064 bytes
in use
2069120:M 27 Sep 2026 14:47:03.857 - DB 0: 2 keys (0 volatile) in 4 slots HT.
2069120:M 27 Sep 2026 14:47:03.857 - 1 clients connected (0 replicas), 812064 bytes
in use

... # 크래시 발생

=== REDIS BUG REPORT START: Cut & paste starting from here ===
2069120:M 27 Sep 2026 14:47:06.795 # Redis 999.999.999 crashed by signal: 11
2069120:M 27 Sep 2026 14:47:06.795 # Crashed running the instruction at: 0x4459b9
2069120:M 27 Sep 2026 14:47:06.795 # Accessing address: 0x7b07f3baa000
2069120:M 27 Sep 2026 14:47:06.795 # Failed assertion: <no assertion failed> (<no
file>:0)

----- STACK TRACE -----
EIP:
./redis/src/redis-server *:6379(stringmatchlen+0x59)[0x4459b9]
```

Backtrace:

```
./redis/src/redis-server *:6379(logStackTrace+0x6b)[0x47a51b]
./redis/src/redis-server *:6379(sigsegvHandler+0xb1)[0x47ac61]
/nix/store/fq4yq8i8wd08xg3fy58l6q73cgy8hjr2-glibc-2.27/lib/
libpthread.so.0(+0x11f10)[0x7b07f3e11f10]
./redis/src/redis-server *:6379(stringmatchlen+0x59)[0x4459b9]
./redis/src/redis-server *:6379(stringmatchlen+0xec)[0x445a4c]
./redis/src/redis-server *:6379(keysCommand+0xe6)[0x44b766]
./redis/src/redis-server *:6379(call+0x90)[0x434740]
./redis/src/redis-server *:6379(processCommand+0x477)[0x435be7]
./redis/src/redis-server *:6379(processInputBuffer+0x179)[0x445439]
./redis/src/redis-server *:6379(aeProcessEvents+0x150)[0x42e350]
./redis/src/redis-server *:6379(aeMain+0x2b)[0x42e78b]
./redis/src/redis-server *:6379(main+0x3bf)[0x42b36f]
/nix/store/fq4yq8i8wd08xg3fy58l6q73cgy8hjr2-glibc-2.27/lib/
libc.so.6(__libc_start_main+0xee)[0x7b07f3a21b8e]
./redis/src/redis-server *:6379(_start+0x2a)[0x42b6ea]
... # 생략
```

redis의 버그 리포트에서 stringmatchlen() 함수를 발견

3 원인

클라이언트가 전달한 패턴과 일치하는 key를 찾는 과정에서

패턴과 string이 일치하는지 확인하는 함수 stringmatchlen()을 호출

함수 stringmatchlen()은 패턴 문자로 NULL문자가 들어가는 걸 상정하지 않았기 때문에 오작동

3.1 과정

3.1.1

변수 상태

```
pattern: ['\0', '*', 'z']
patternlen: 3
string: "\0" # null terminated string
stringlen: 0
```

실행 코드

```
int stringmatchlen(const char *pattern, int patternLen,
                  const char *string, int stringLen, int nocase)
{
    while(patternLen) {
        switch(pattern[0]) {
            case '*':
... # 생략
            default:
                if (!nocase) {
                    if (pattern[0] != string[0])
                        return 0; /* no match */
                } else {
... # 생략
                }
                string++;
                stringLen--;
                break;
        }
        pattern++;
        patternLen--;
    }
```

if (pattern[0] != string[0]) 둘다 NULL이기 때문에 조건문에 들어가지 못하고 계속 실행

string은 주어진 메모리 영역을 벗어남

while(patternLen)에 의해 반복

3.1.2

변수 상태

```
Pattern: ['*', 'z']
patternlen: 2
string: 잘못된 주소
stringlen: -1
```

실행 코드

```
int stringmatchlen(const char *pattern, int patternLen,
                  const char *string, int stringLen, int nocase)
{
    while(patternLen) {
        switch(pattern[0]) {
            case '*':
... # 생략
                if (patternLen == 1)
                    return 1; /* match */
                while(stringLen) {
                    if (stringmatchlen(pattern+1, patternLen-1,
                                        string, stringLen, nocase))
```

stringLen이 -1이기 때문에 while(stringLen)에 들어가서 재귀 호출

3.1.3

변수 상태

```
Pattern: ['z']
patternlen: 1
string: 잘못된 주소
stringlen: -1
```

실행 코드

```
int stringmatchlen(const char *pattern, int patternLen,
                  const char *string, int stringLen, int nocase)
{
    while(patternLen) {
        switch(pattern[0]) {
            case '*':
... # 생략
            default:
                if (!nocase) {
                    if (pattern[0] != string[0])
```

string[0]에서 잘못된 주소 접근으로 인한 segmentation fault

4 수정

```
int stringmatchlen(const char *pattern, int patternLen,
                  const char *string, int stringLen, int nocase)
{
-   while(patternLen) {
+   while(patternLen && stringLen) {
        switch(pattern[0]) {
            case '*':
                while (pattern[1] == '*') {
```

반복문에 stringLen 조건을 추가하여 string의 마지막 부분인 NULL이 비교되는 상황을 원천 차단